

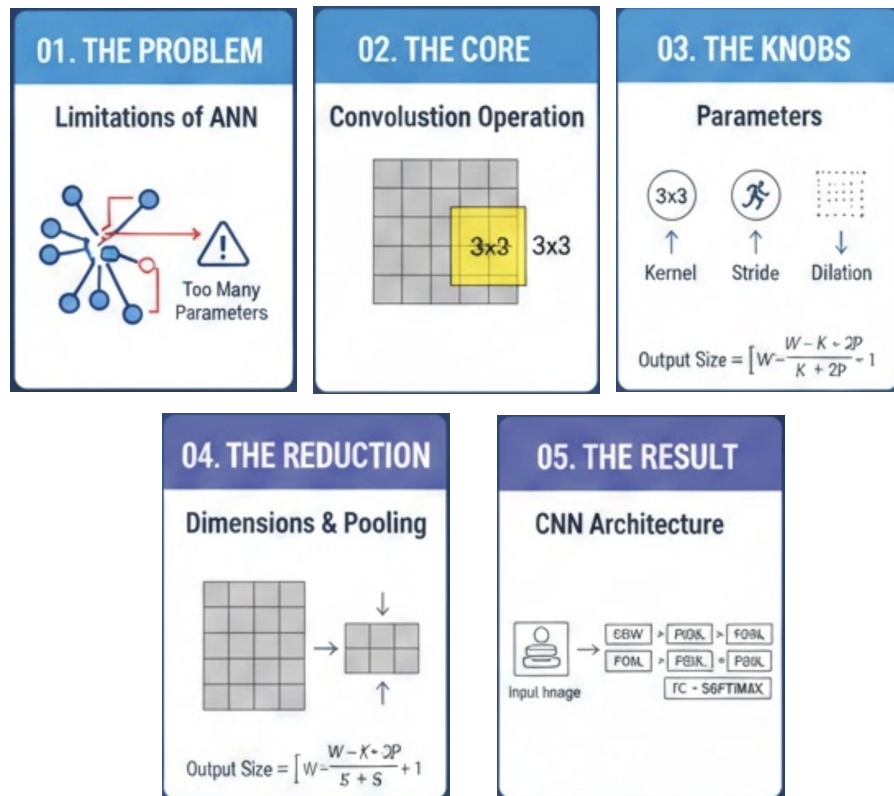
Join the lecture online on your dashboard.

Advanced CNN Architectures and applications

Recap

Last lecture we covered:

1. Upsampling
2. AlexNet/VGG
3. Residual Blocks and Skip Connections
4. ResNet
5. Applications



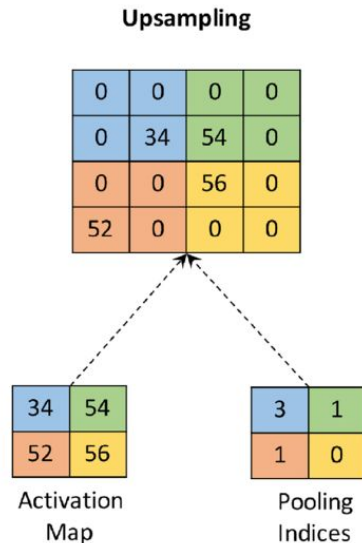
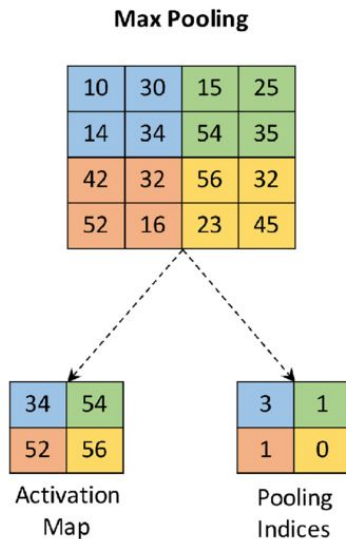
Recap

Upsampling

Upsampling is a technique used to **increase the spatial resolution** of feature maps, effectively transforming low-resolution data into a higher-dimensional representation.

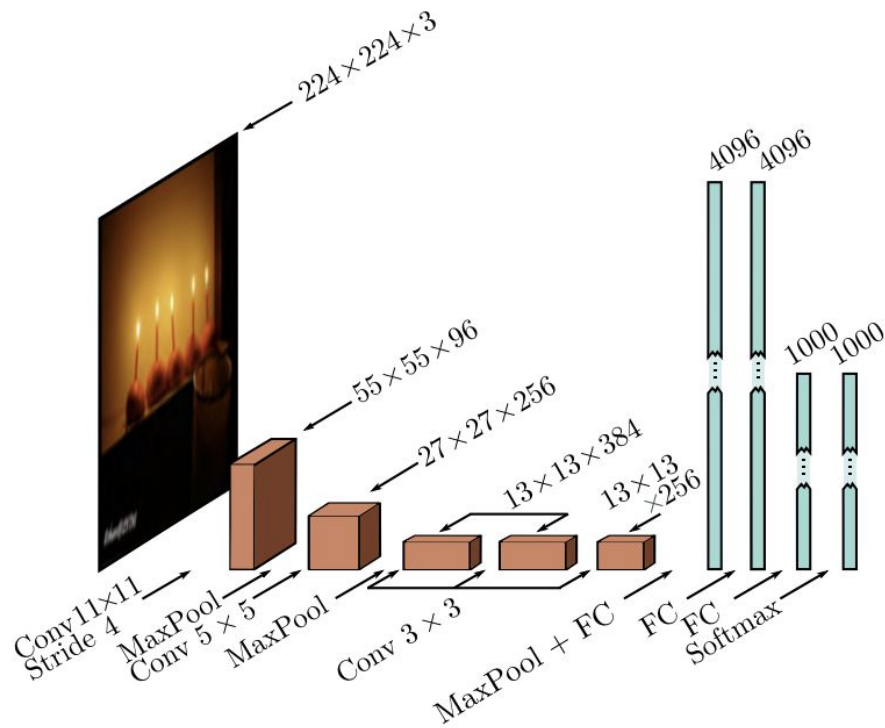
Common Methods:

- **Nearest Neighbor / Bilinear Interpolation:**
Simple, non-learnable techniques that fill new pixels based on surrounding values.
- **Transposed Convolution (Deconvolution):** A learnable layer that uses weights to optimally reconstruct spatial detail during training.



AlexNet

Figure 10.16 AlexNet (Krizhevsky et al., 2012). The network maps a 224×224 color image to a 1000-dimensional vector representing class probabilities. The network first convolves with 11×11 kernels and stride 4 to create 96 channels. It decreases the resolution again using a max pool operation and applies a 5×5 convolutional layer. Another max pooling layer follows, and three 3×3 convolutional layers are applied. After a final max pooling operation, the result is vectorized and passed through three fully connected (FC) layers and finally the softmax layer.



VGG

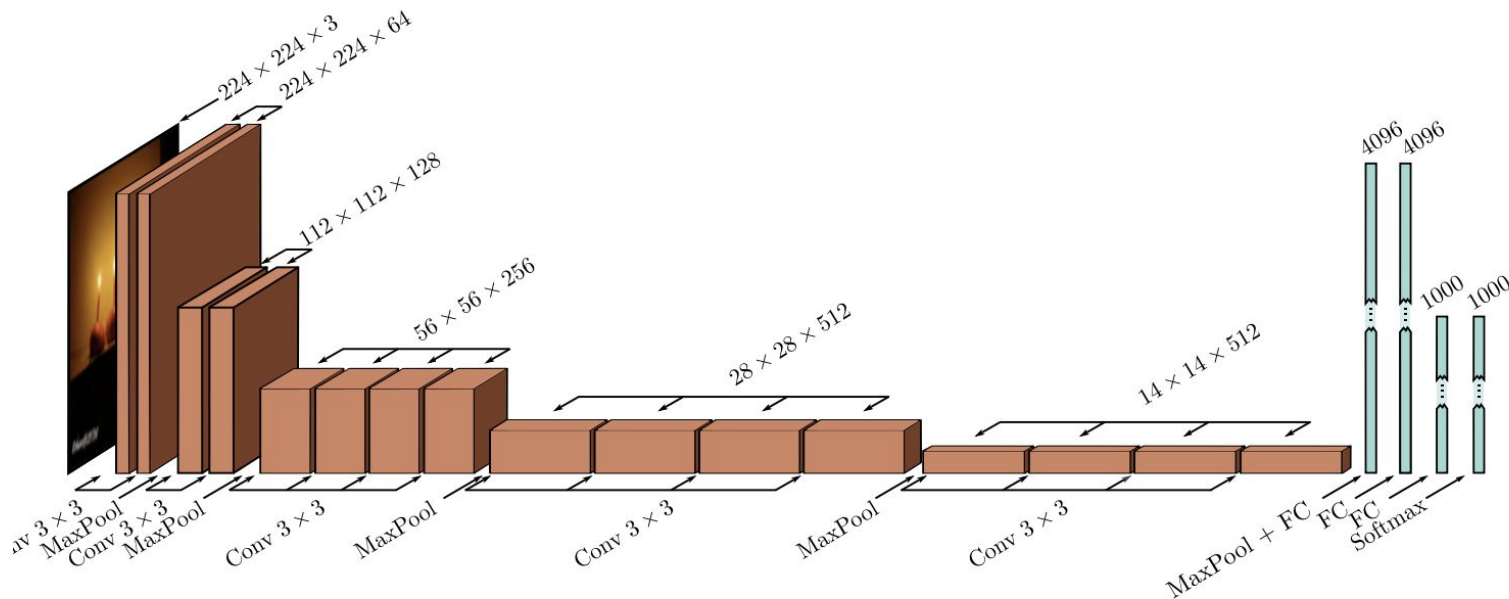


Figure 10.17 VGG network (Simonyan & Zisserman, 2014) depicted at the same scale as AlexNet (see figure 10.16). This network consists of a series of convolutional layers and max pooling operations, in which the spatial scale of the representation gradually decreases, but the number of channels gradually increases. The hidden layer after the last convolutional operation is resized to a 1D vector and three fully connected layers follow. The network outputs 1000 activations corresponding to the class labels that are passed through a softmax function to create class probabilities.

Residual Blocks and Skip Connections

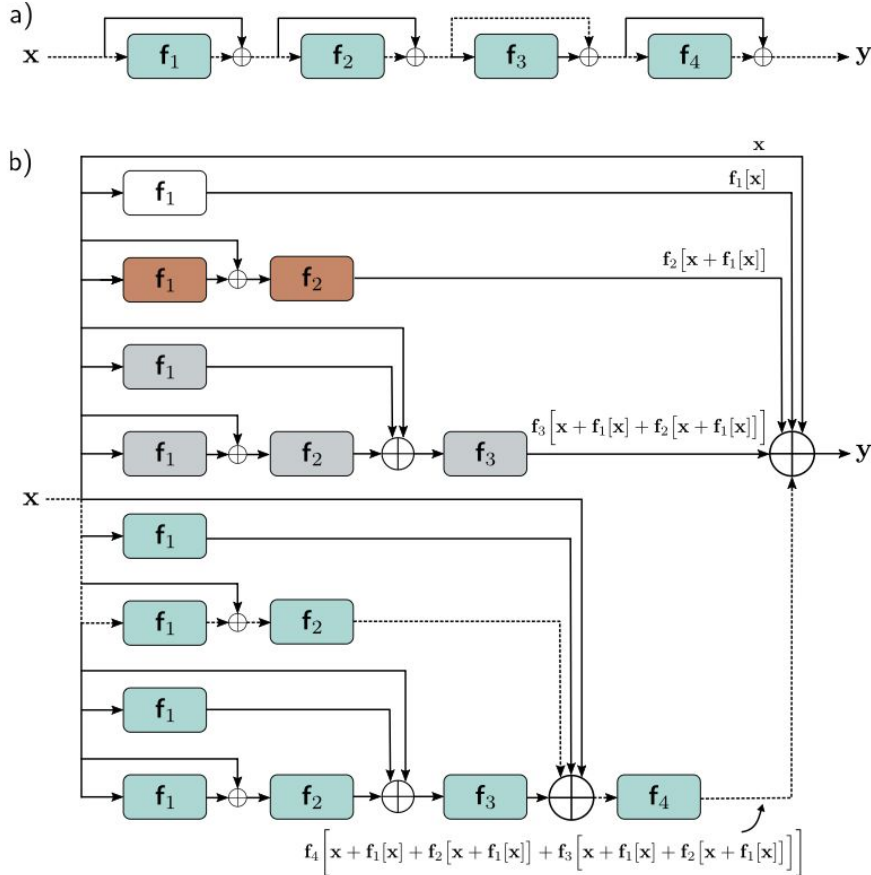


Figure 11.4 Residual connections.

a) The output of each function $f_k[x, \phi_k]$ is added back to its input, which is passed via a parallel computational path called a residual or skip connection. Hence, the function computes an additive change to the representation.

b) Upon expanding (unraveling) the network equations, we find that the output is the sum of the input plus four smaller networks (depicted in white, orange, gray, and cyan, respectively, and corresponding to terms in equation 11.5);

we can think of this as an ensemble of networks. Moreover, the output from the cyan network is itself a transformation $f_4[\cdot, \phi_4]$ of another ensemble, and so on. Alternatively, we can consider the network as a combination of **16 different paths** through the computational graph. One example is the dashed path from input x to output y , which is the same in panels (a) and (b).

ResNet

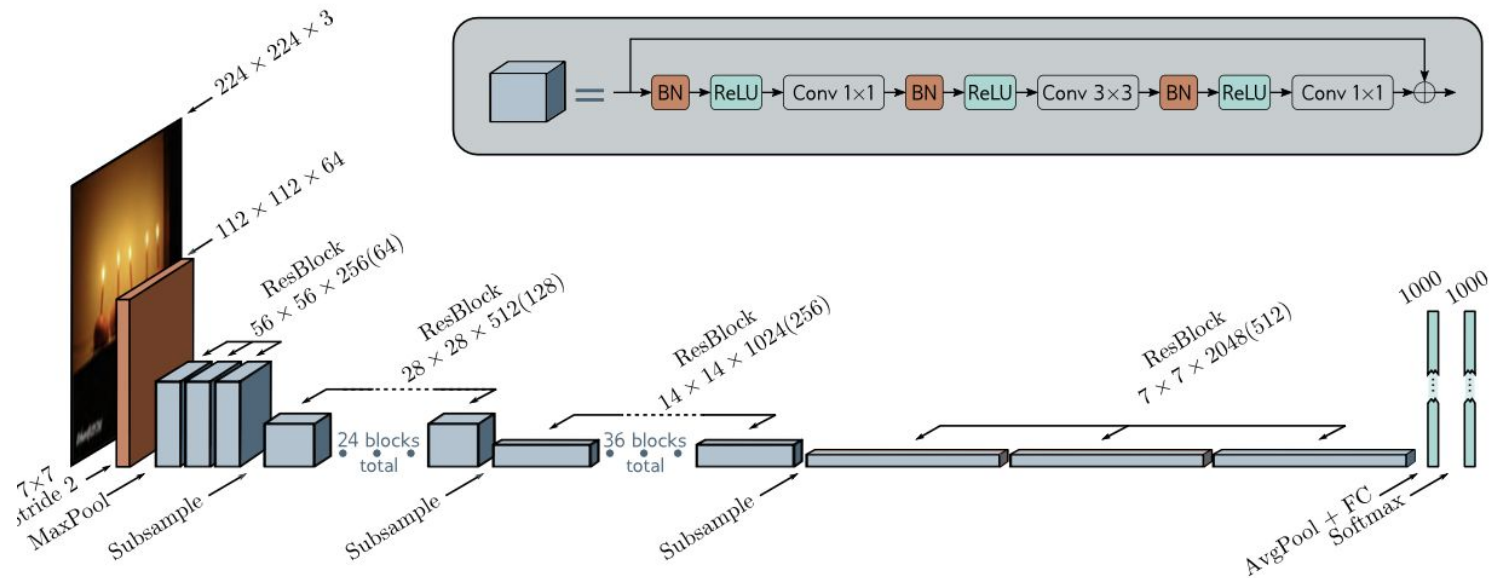
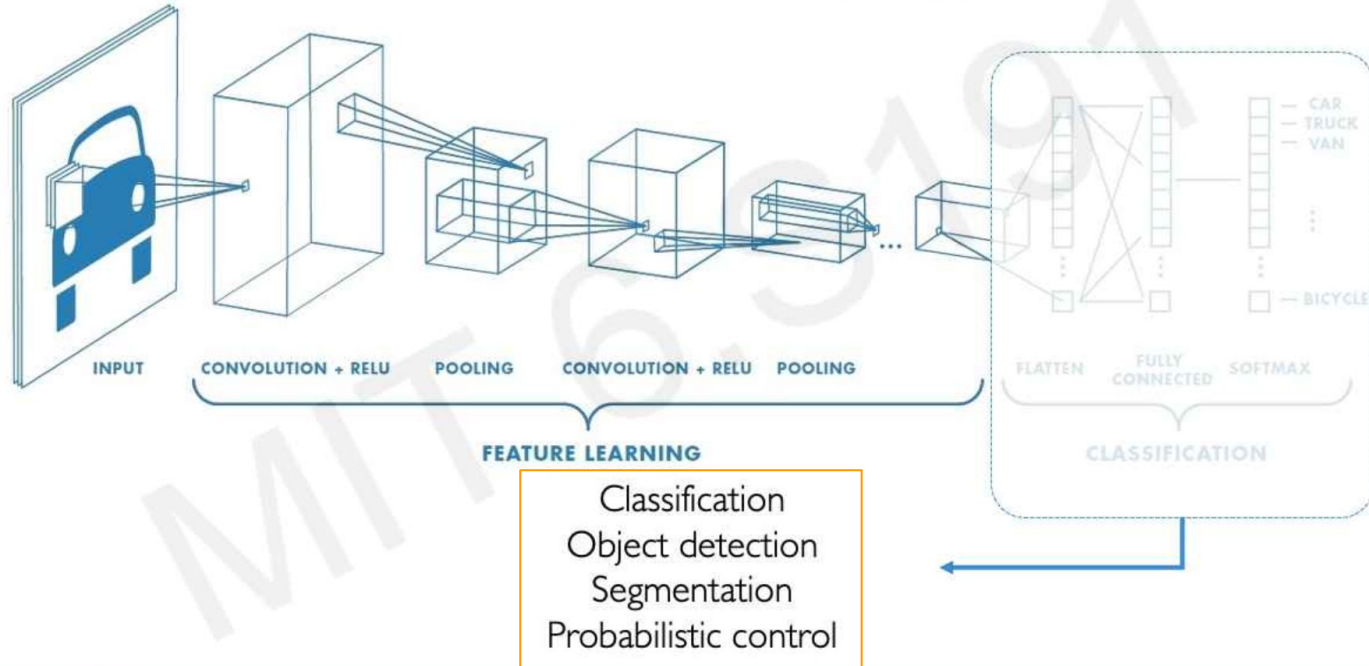


Figure 11.8 ResNet-200 model. A standard 7×7 convolutional layer with stride two is applied, followed by a MaxPool operation. A series of bottleneck residual blocks follow (number in brackets is channels after first 1×1 convolution), with periodic downsampling and accompanying increases in the number of channels. The network concludes with average pooling across all spatial positions and a fully connected layer that maps to pre-softmax activations.

Applications

An Architecture for Many Applications



Quiz

What is a primary difference between AlexNet and VGG-16?

- A) AlexNet is significantly deeper than VGG-16.
- B) VGG-16 uses larger convolutional kernels (like 11 x 11) compared to AlexNet.
- C) VGG-16 increased the network depth (up to 19 layers) and used 3 x 3 kernels to achieve better performance.
- D) AlexNet was the first to achieve a top-5 error rate below 10%.

VGG increased depth to 19 layers
vs AlexNet's 8

Quiz

What is the "Shattered Gradient" problem in deep sequential networks?

A) Gradients become too large and cause numerical overflow.

B) The correlation between input and gradient drops to zero, making the loss surface look like chaotic white noise.

C) The network runs out of memory due to too many parameters.

D) The kernels stop extracting local features correctly.

The correlation between input and gradient drops to zero, making the loss surface look like chaotic white noise.

Quiz

Why is Batch Normalization considered an "enabler" for deep ResNets?

- A) It reduces the number of parameters needed in a 3x3 convolution.
- B) It prevents the exponential growth of variance that occurs when adding the input back to the output in residual blocks.
- C) It automatically calculates the correct stride for every layer.
- D) It eliminates the need for any skip connections.

Without Batch Normalization, variance doubles at each layer, growing 2^K .

Agenda

Today we will cover:

1. U-Net
2. Inception Network
 - a. Concept
 - b. Evolution
 - c. Comparison
 - d. Shortcoming
3. R-CNN
4. YoLo
 - a. Conceptual
 - b. SOTA
5. SAM

• Architectures:

- U-Net 
- Inception Network 
 - Concept
 - Evolution
 - Comparison
 - Shortcoming

• Object Detection:

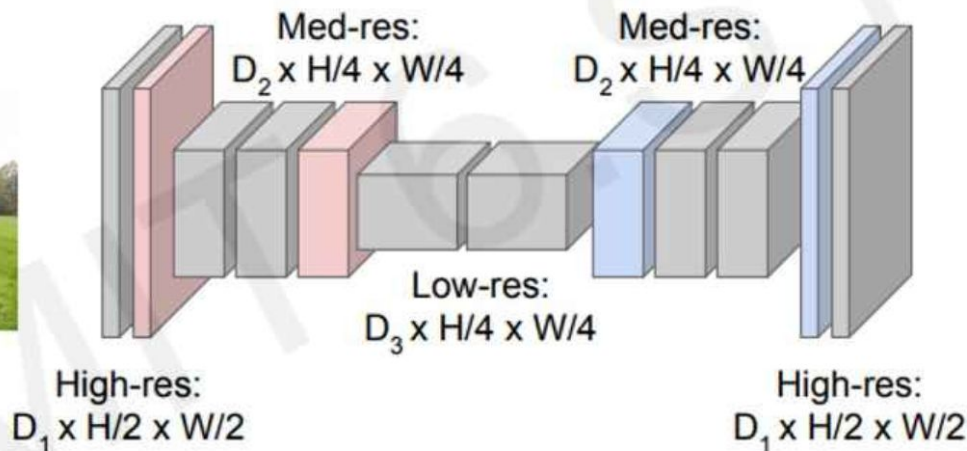
- R-CNN 
- YoLo 
 - Conceptual
 - SOTA

Semantic Segmentation: Fully Convolutional Networks

FCN: Fully Convolutional Network.
Network designed with all convolutional layers,
with **downsampling** and **upsampling** operations



Input:
 $3 \times H \times W$



Predictions:
 $H \times W$



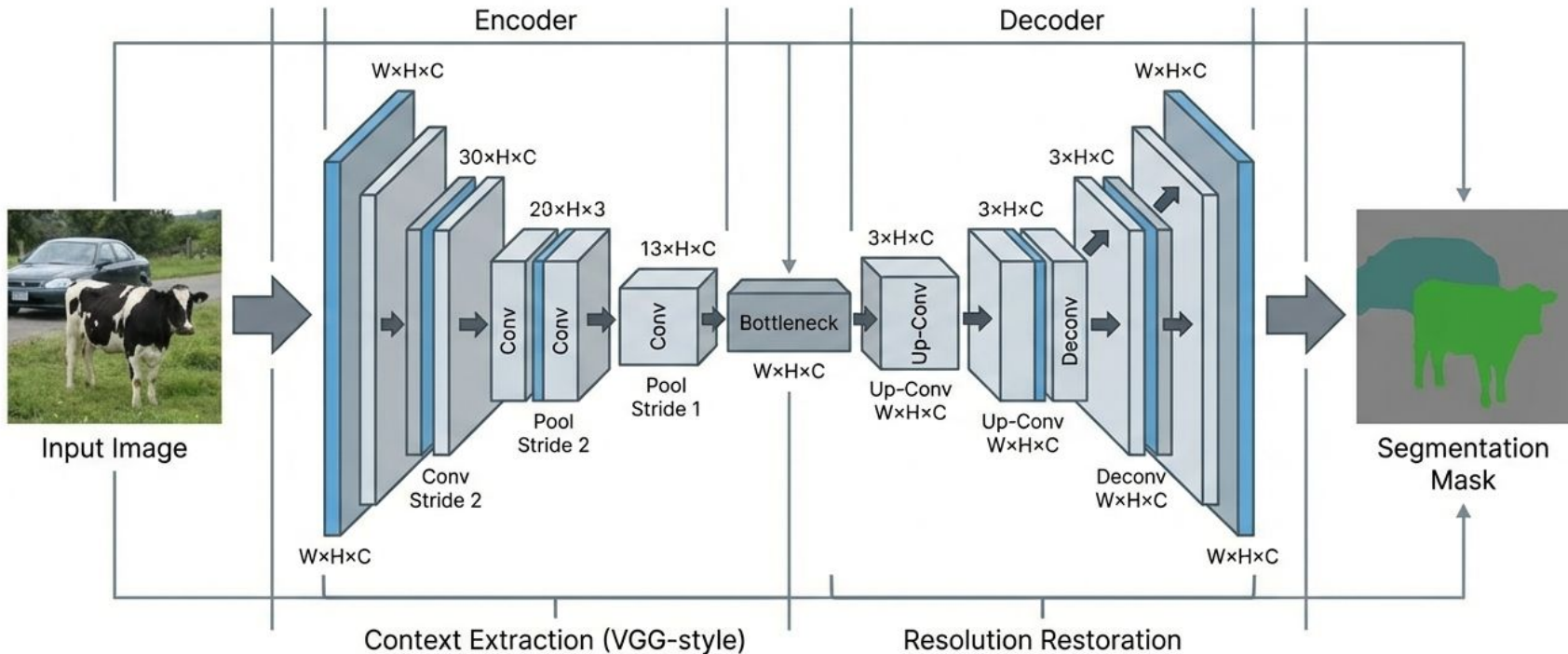
`tf.keras.layers.Conv2DTranspose`



`torch.nn.ConvTranspose2d`

Application: Semantic Segmentation

Goal: Assign a class label to every pixel.



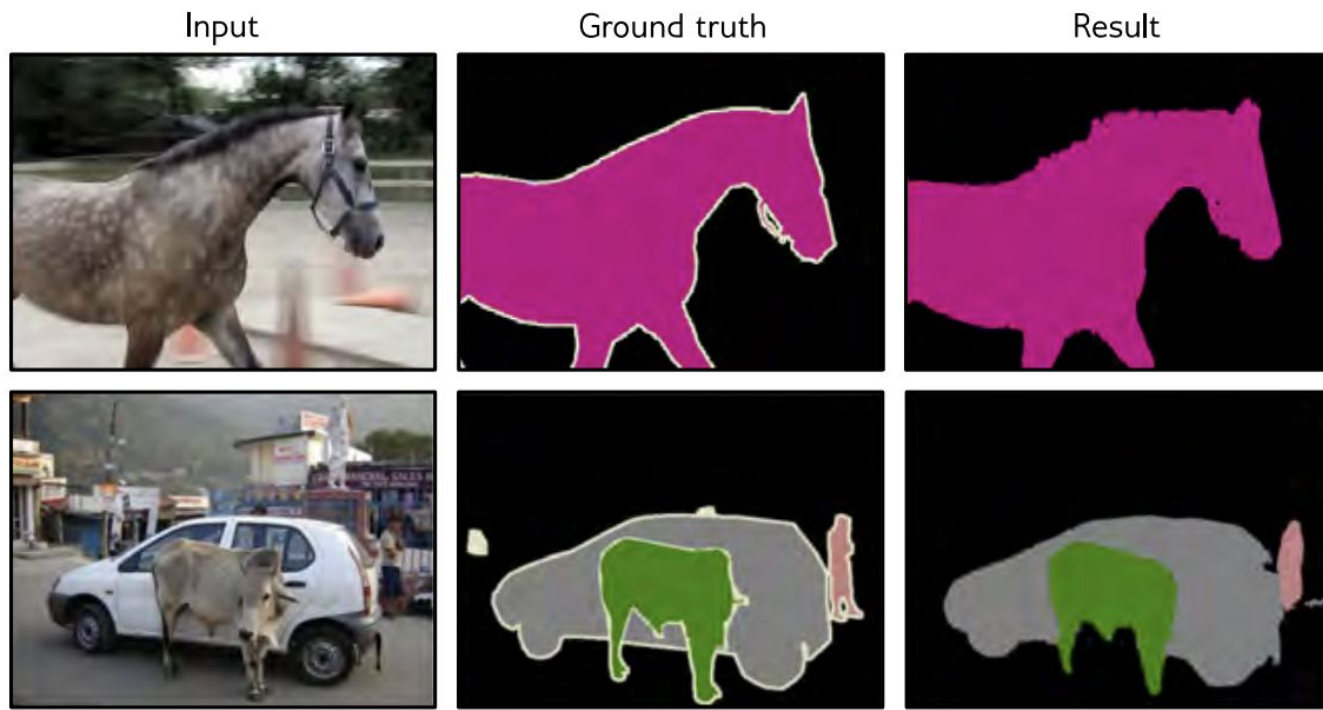
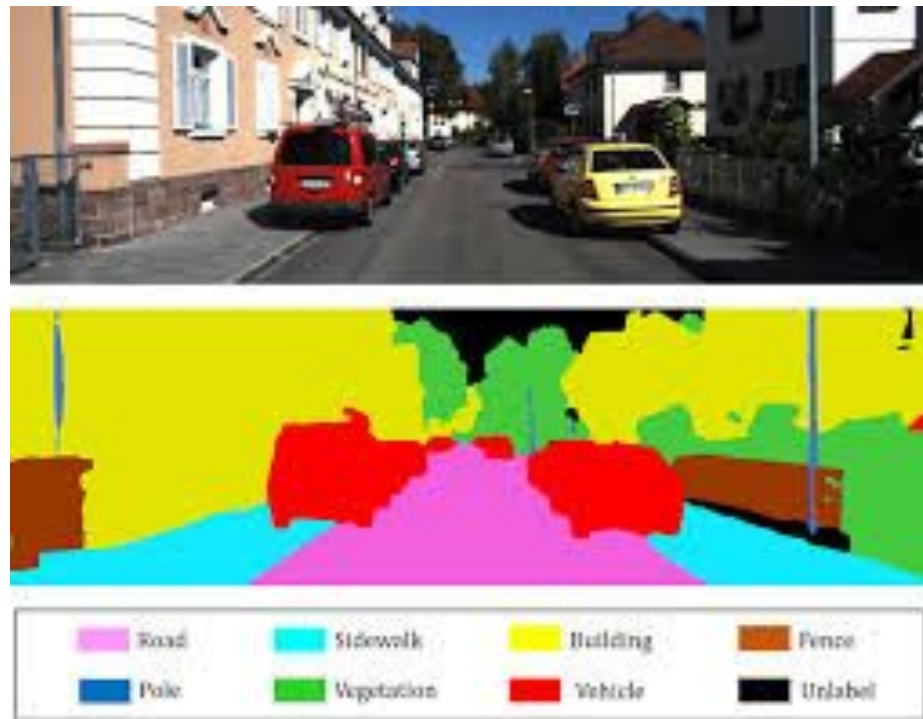


Figure 10.20 Semantic segmentation results. The final result is created from the probability maps by greedily selecting the best class and using a heuristic method to find a sensible binary map based on the probabilities and their spatial proximity. If there is enough evidence, subsequent classes are added, and their segmentation maps are combined. Adapted from Noh et al. (2015).

Shortcoming of ResNet

The "shortcoming" of ResNet was not a failure in its own logic, but its inability to handle pixel-perfect localization required for tasks like **semantic segmentation**.

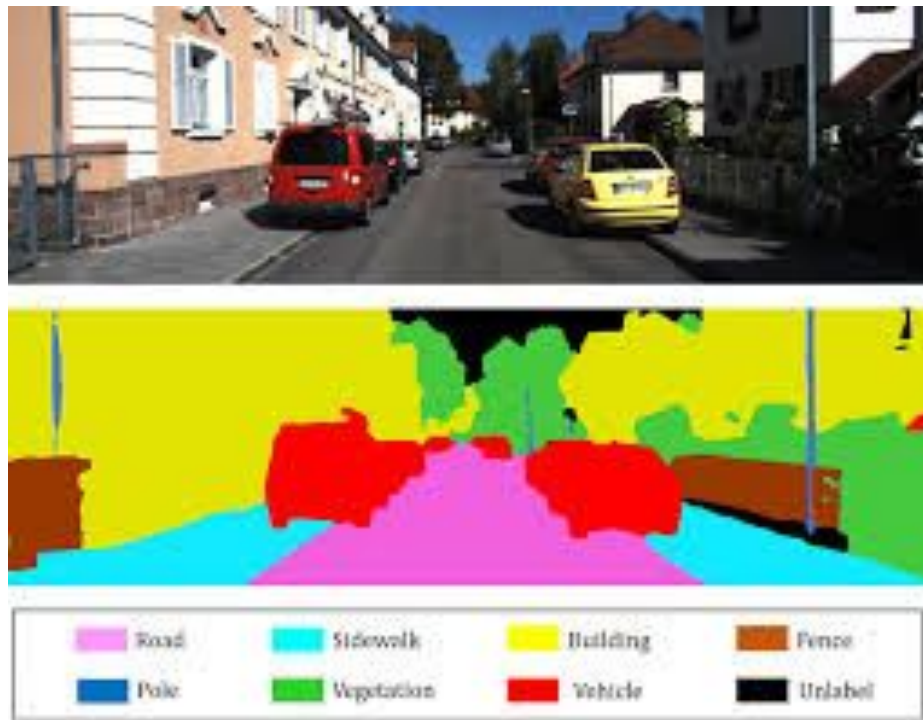
Therefore U-Net was created for tasks where you need to know "where is it" at a pixel level (e.g., medical imaging).



U-Net : Introduction

Origin: Introduced by
Ronneberger et al. (2015) for
biomedical image segmentation

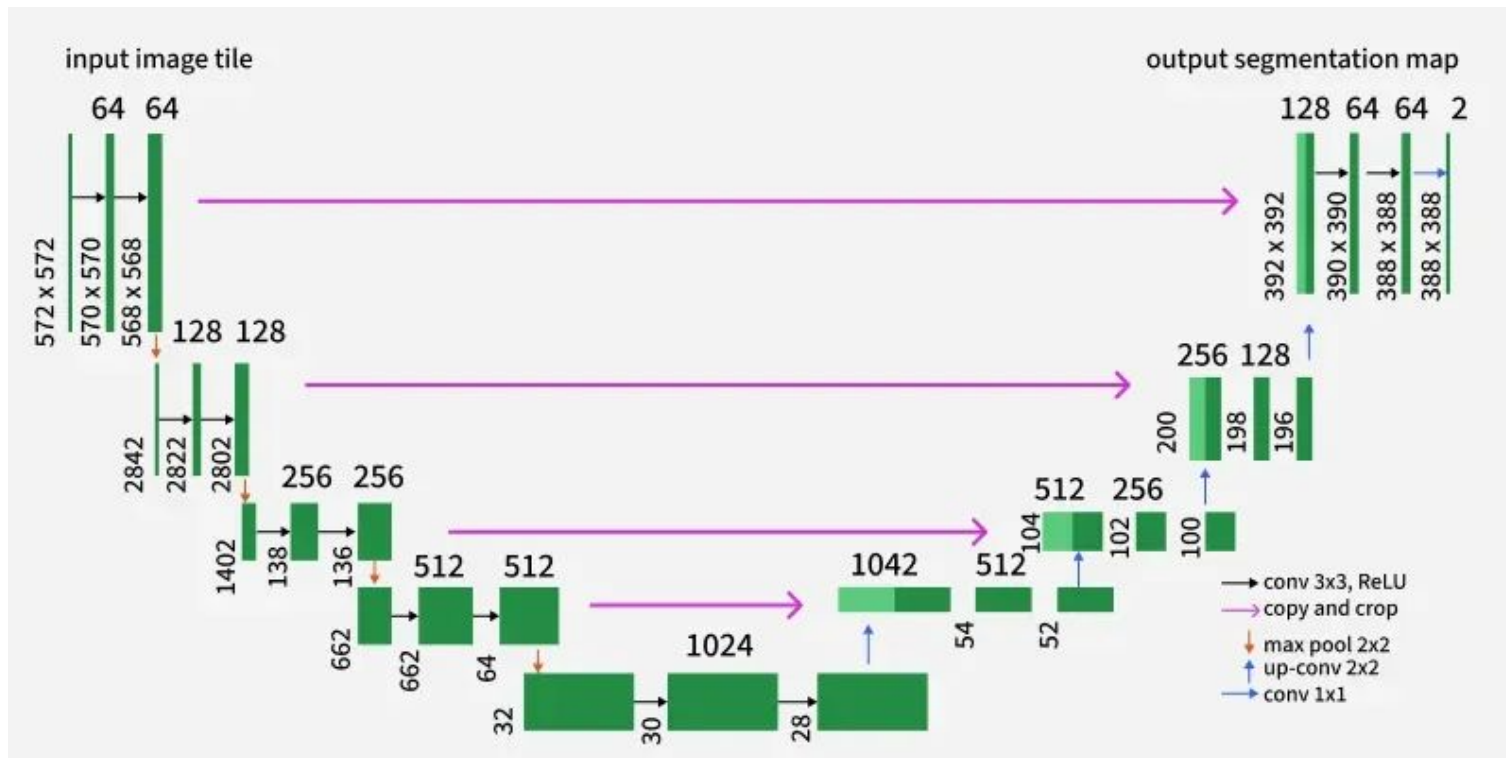
Purpose: Designed specifically
for semantic segmentation,
where the goal is to classify
every individual pixel in an
image rather than just the image
as a whole



U-Net : Architecture

- U-Shaped Structure: The network consists of a symmetrical arrangement of two paths, giving it the distinctive "U" shape for which it is named.
 - Encoder (Down-sampling): Reduces spatial resolution using strided convolutions or pooling to extract high-level, semantic features (the "what").
 - Decoder (Up-sampling): Increases dimensionality back to the original image resolution to recover positional information (the "where")

U-Net : Architecture



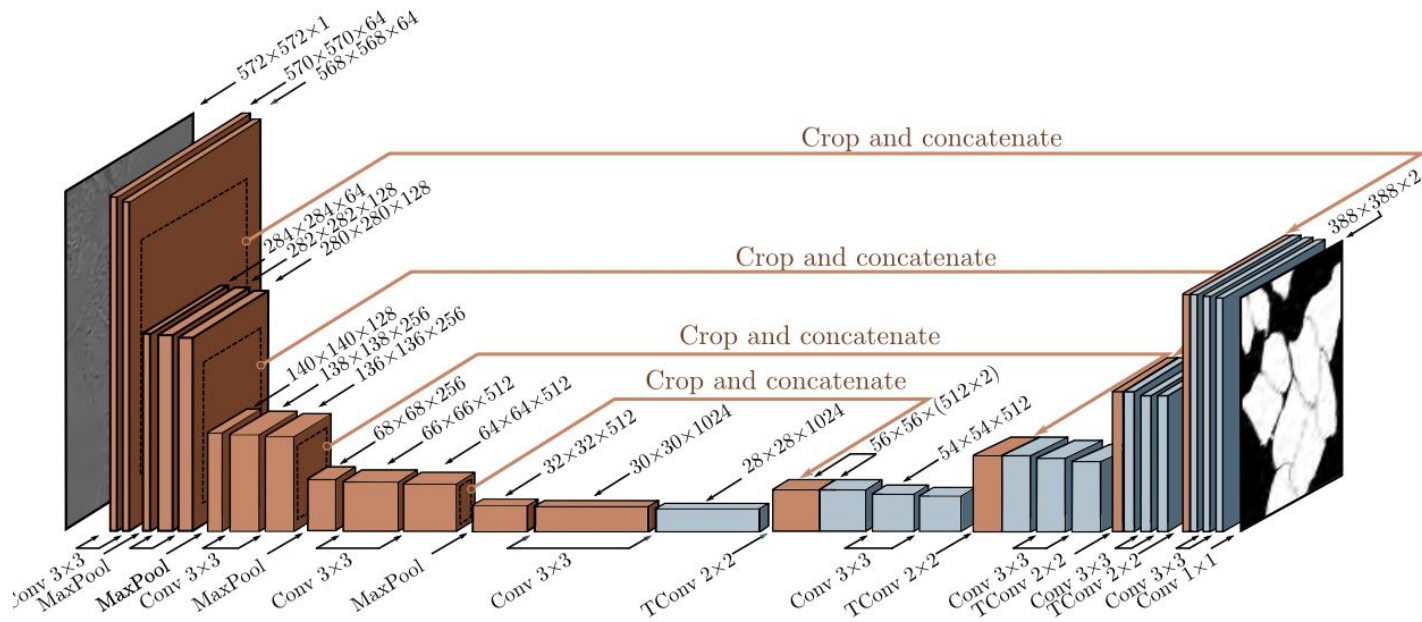


Figure 11.10 U-Net for segmenting HeLa cells. The U-Net has an encoder-decoder structure, in which the representation is downsampled (orange blocks) and then re-upscaled (blue blocks). The encoder uses regular convolutions, and the decoder uses transposed convolutions. Residual connections append the last representation at each scale in the encoder to the first representation at the same scale in the decoder (orange arrows). The original U-Net used “valid” convolutions, so the size decreased slightly with each layer, even without downsampling. Hence, the representations from the encoder were cropped (dashed squares) before appending to the decoder. Adapted from Ronneberger et al. (2015).

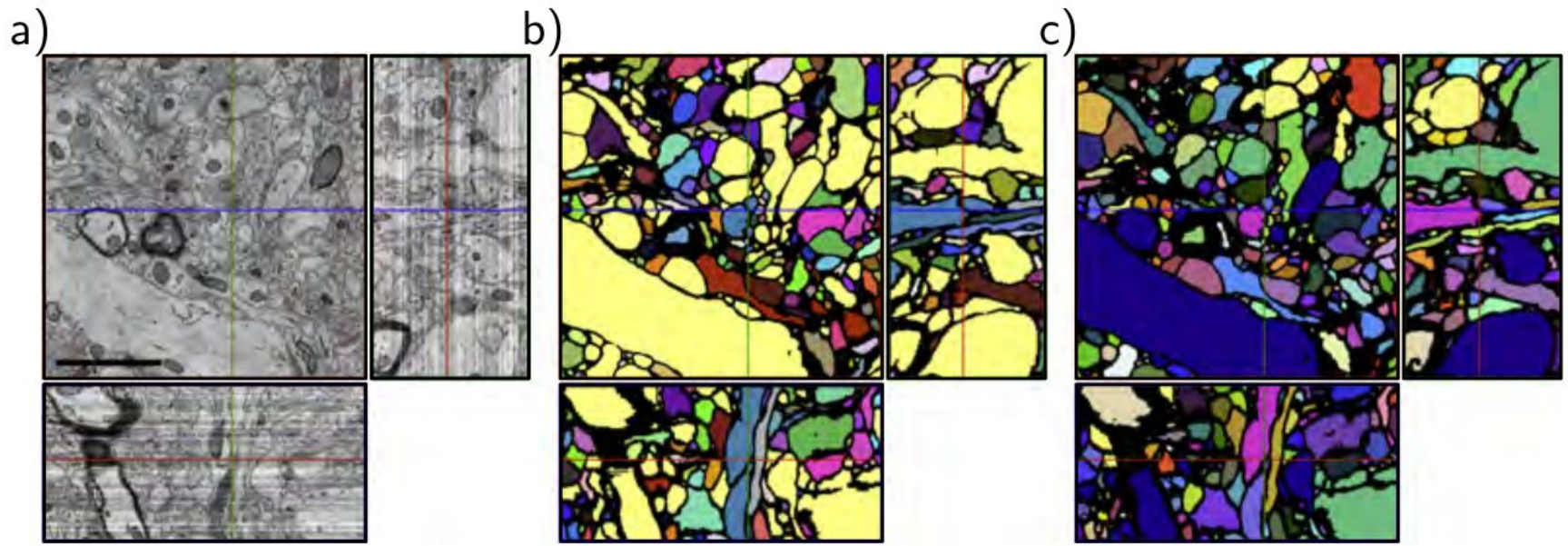
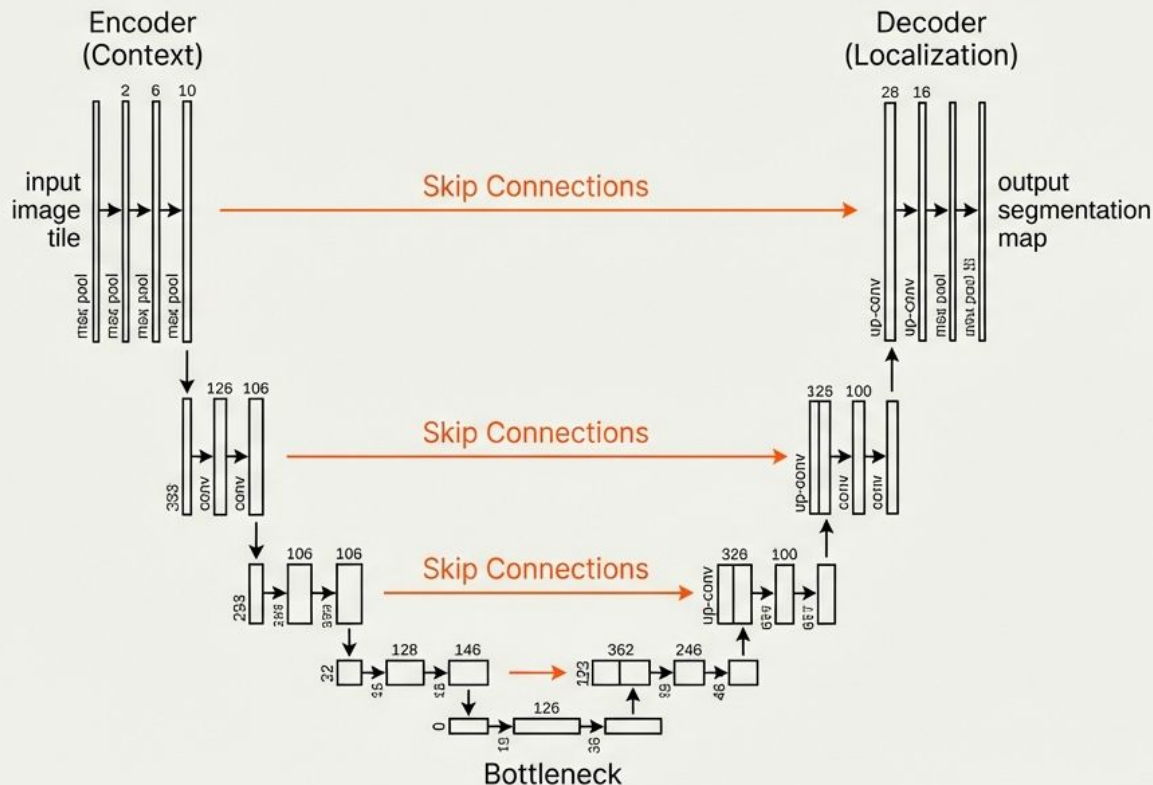


Figure 11.11 Segmentation using U-Net in 3D. a) Three slices through a 3D volume of mouse cortex taken by scanning electron microscope. b) A single U-Net is used to classify voxels as being inside or outside neurites. Connected regions are identified with different colors. c) For a better result, an ensemble of five U-Nets is trained, and a voxel is only classified as belonging to the cell if all five networks agree. Adapted from Falk et al. (2019).

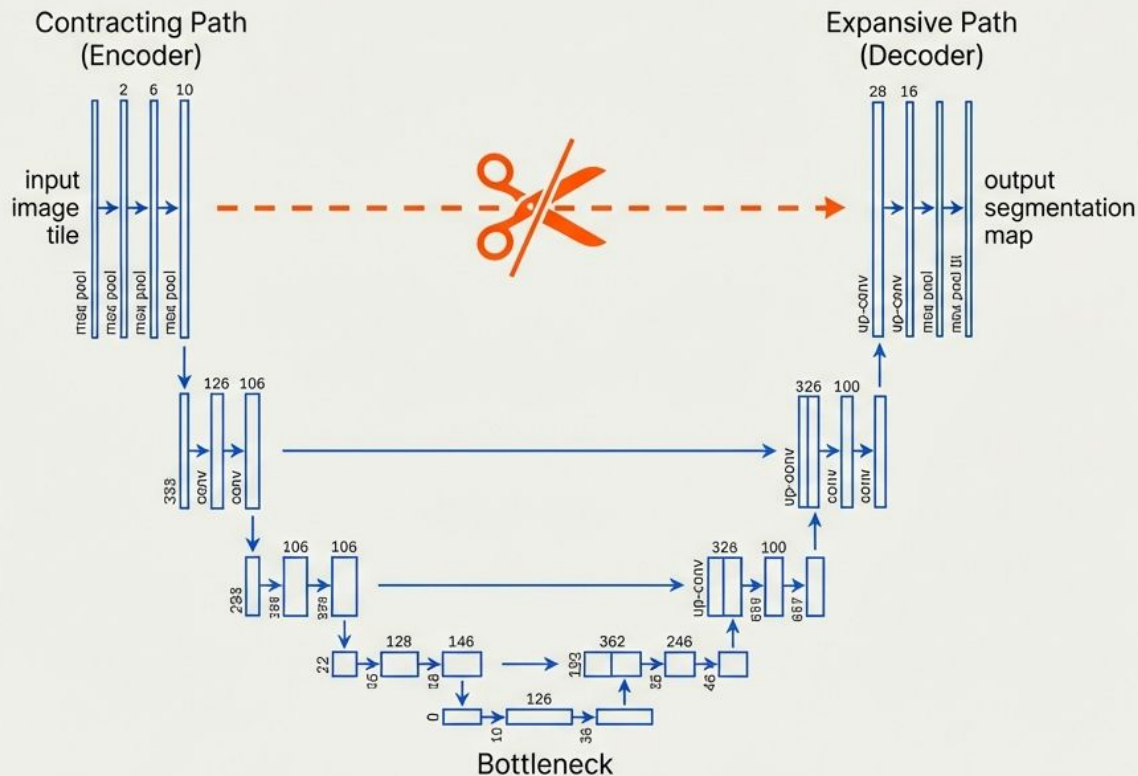
U-Net: Solving the Localization vs. Context Conflict



Standard CNNs discard spatial resolution to gain semantic depth. U-Net preserves both via a symmetric Encoder-Decoder structure.

1. **Contracting Path:** Captures context (3x3 conv).
2. **Expansive Path:** Enables localization (Up-conv).
3. **Innovation:** Long-range Skip Connections recover high-frequency spatial details lost during downsampling.

2024 Analysis: The Liability of the Uppermost Skip Connection



Current findings suggest removing the top skip connection improves Out-of-Distribution (OOD) robustness.

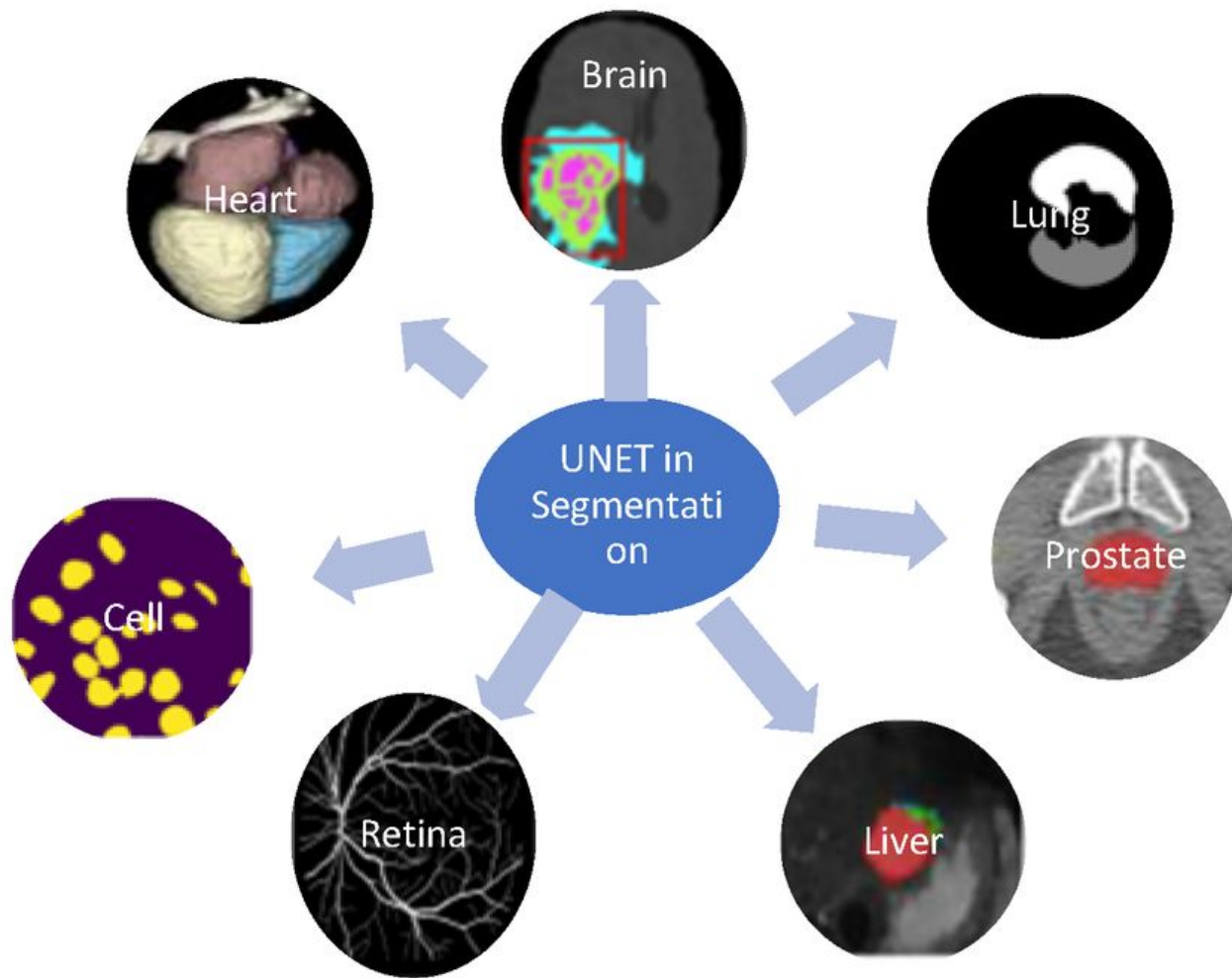
Metric: Domain Susceptibility (Hellinger Distance)

Impact:

+10% Performance (In-Domain)

+13% Performance (Cross-Domain/OOD)

Shallow layers transmit raw noise/artifacts. Severing this link forces the model to rely on processed, semantic features.



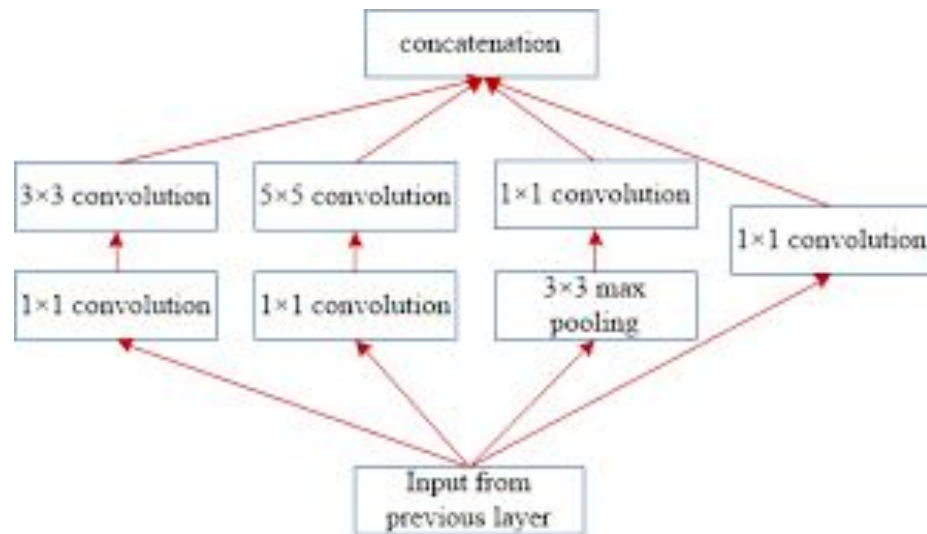
Application:

Biomedical
segmentation

Inception Network

The Inception network is a influential family of Convolutional Neural Network architectures introduced by Google researchers in 2014.

It shifted the deep learning paradigm from simply stacking layers deeper to building wider networks that process information at multiple scales simultaneously.



Core Innovation: The Inception Module

Inception Module performs several different convolutional and pooling operations in parallel rather than sequentially.

Parallel Filters: A typical module applies 1x1, 3x3, and 5x5 convolutions along with 3x3 max pooling all at once on the same input.

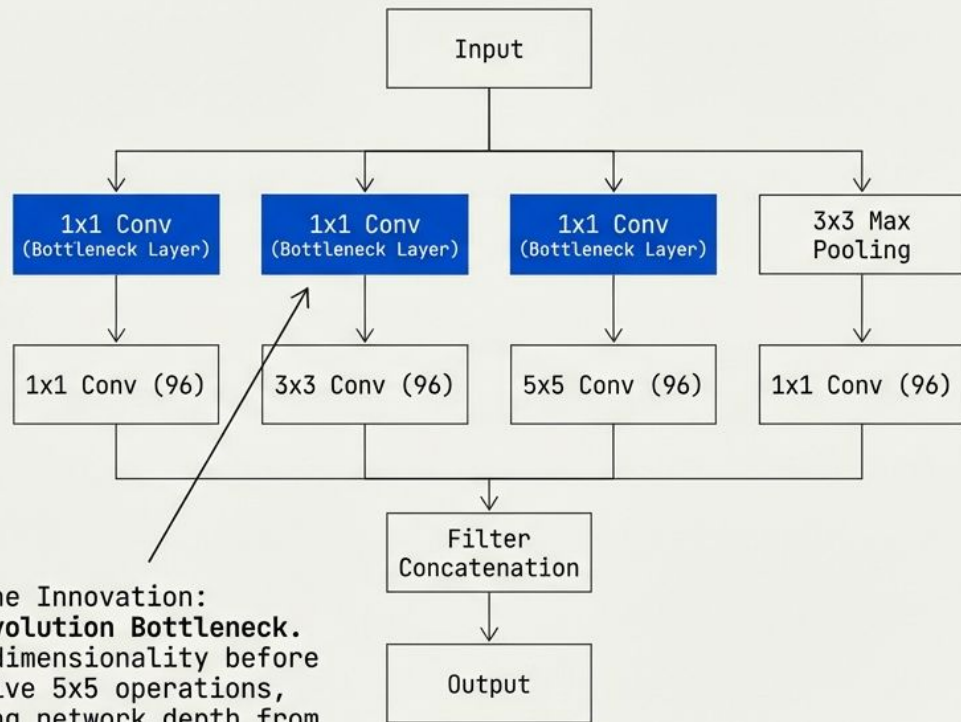
Multi-scale Feature Extraction: This design allows the network to capture both fine-grained details and larger, more abstract patterns in a single layer.

1x1 Convolutions (Bottleneck Layers): To manage computational costs, "bottleneck" 1x1 convolutions are used to reduce the number of channels before applying more expensive 3x3 or 5x5 filters.

Inception (v1): The Multi-Scale Extraction Paradigm

Problem: Salient features vary in size (local vs. global).

Solution: Run multiple filter sizes in parallel.



The Innovation:
1x1 Convolution Bottleneck.
Reduces dimensionality before
expensive 5x5 operations,
decoupling network depth from
computational cost.

Inception-v1 / GoogLeNet

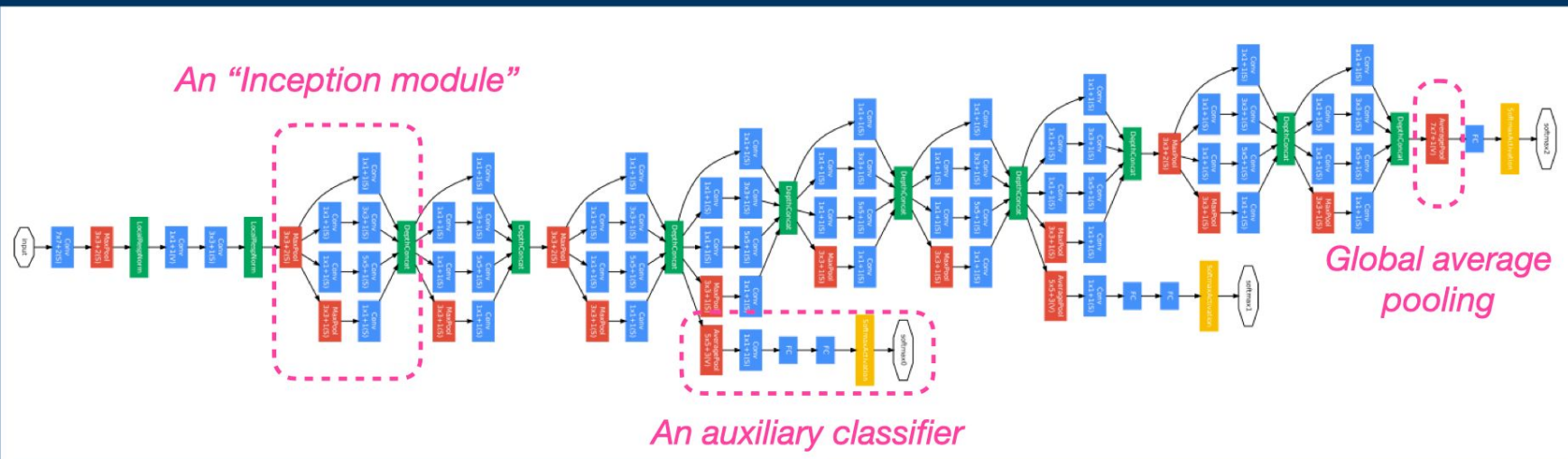
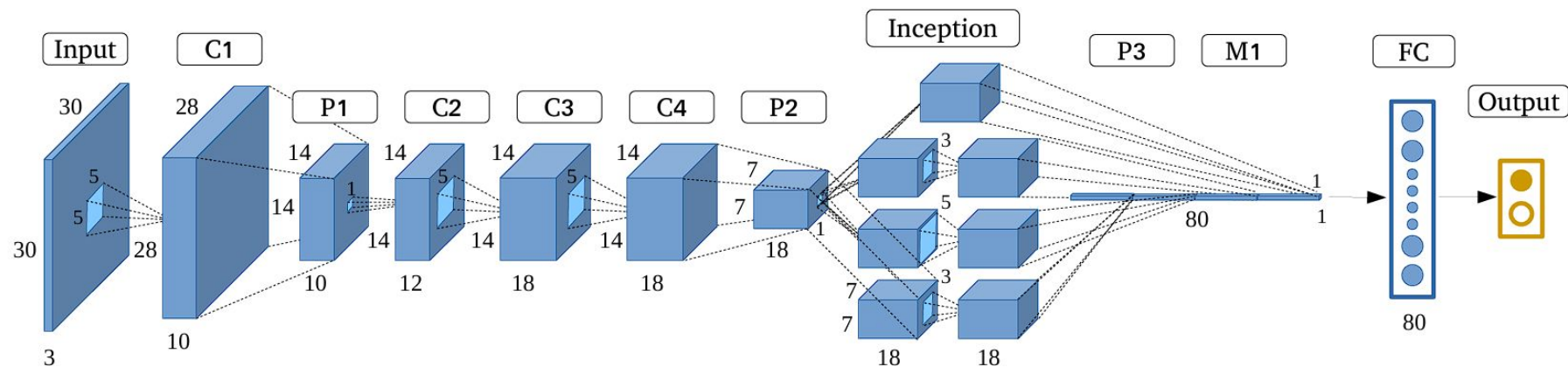


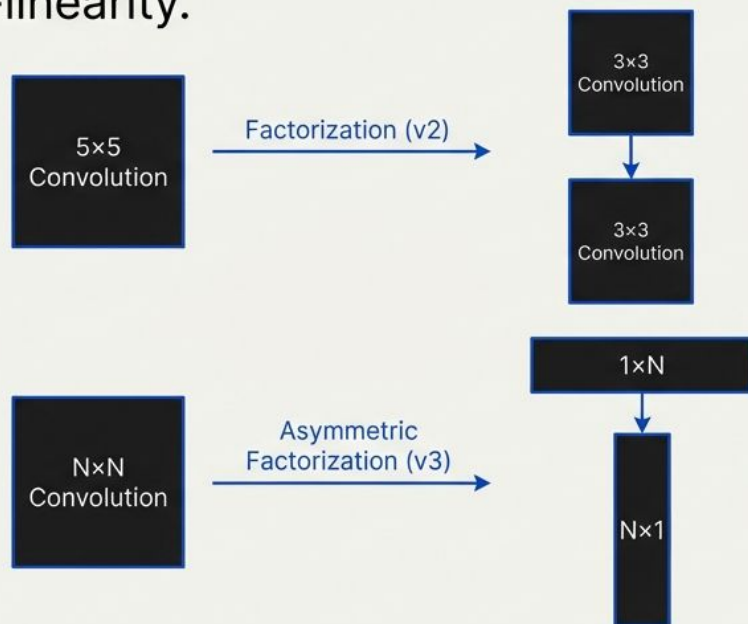
Image modified from: Szegedy, Christian, et al. "Going deeper with convolutions." Proceedings of the IEEE conference on computer vision and pattern recognition. 2015.

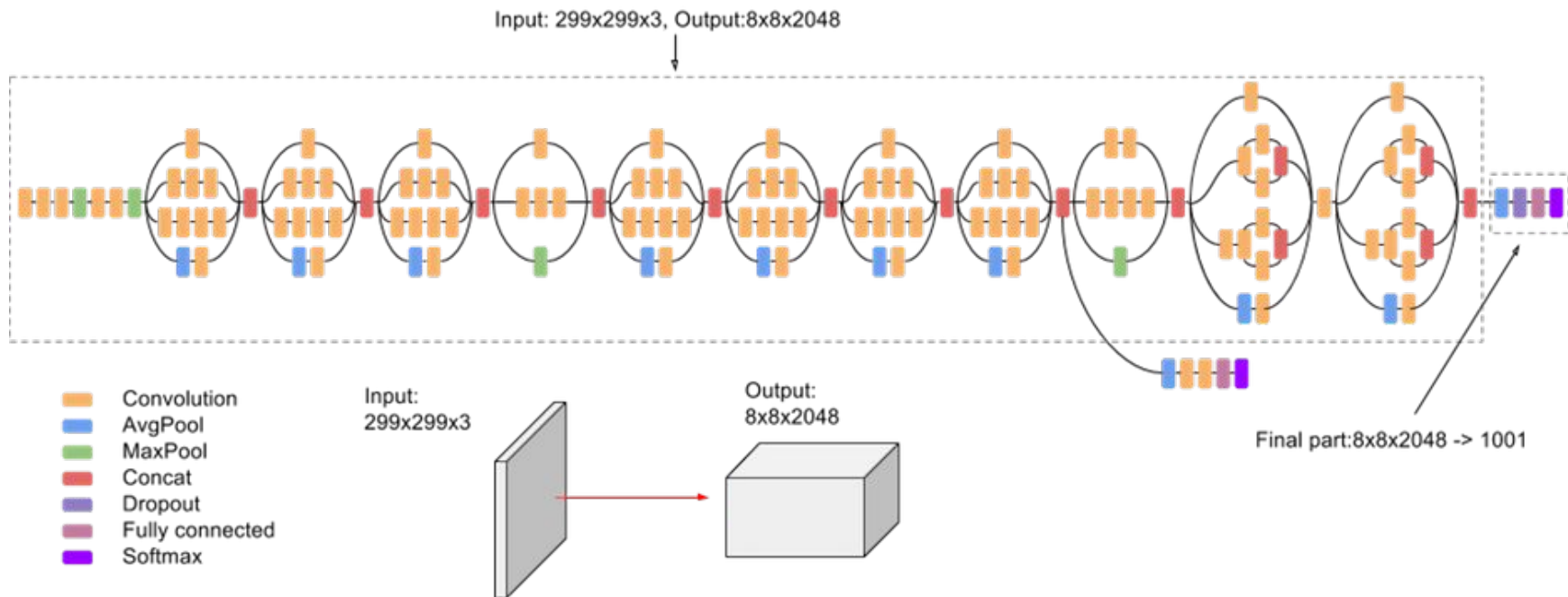


Inception v1 simplified

Inception v2 & v3: The Logic of Factorization

Replacing one 5×5 with two 3×3 s reduces parameters by **~28%** while maintaining **the receptive field**. **Asymmetric factorization** ($1\times n + n\times 1$) further increases non-linearity.

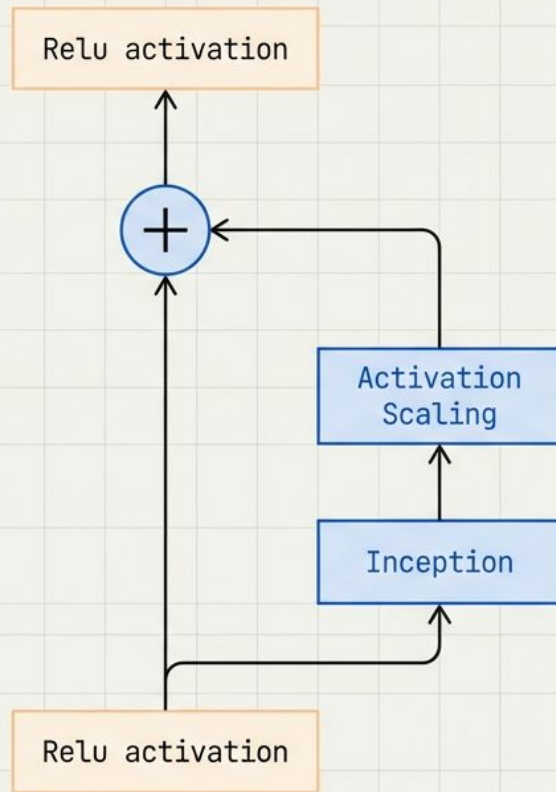




Inception v3 architecture

Inception-v4 & ResNet: Uniformity and Convergence

1. **Uniformity:** Inception-v4 removed "historical baggage" (complex structures from old hardware constraints), standardizing modules.
2. **Residuals:** Inception-ResNet replaces concatenation with **summation**. Result: Dramatically faster training convergence.

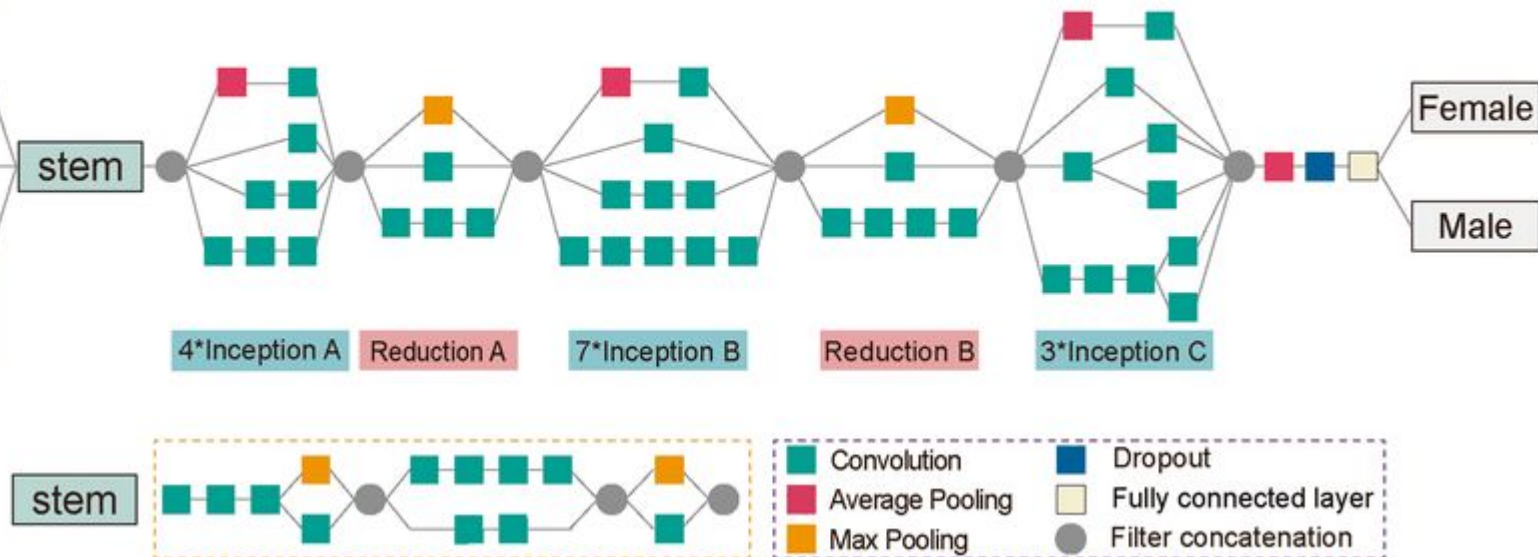


Pelvic images



GoogLeNet Inception V4

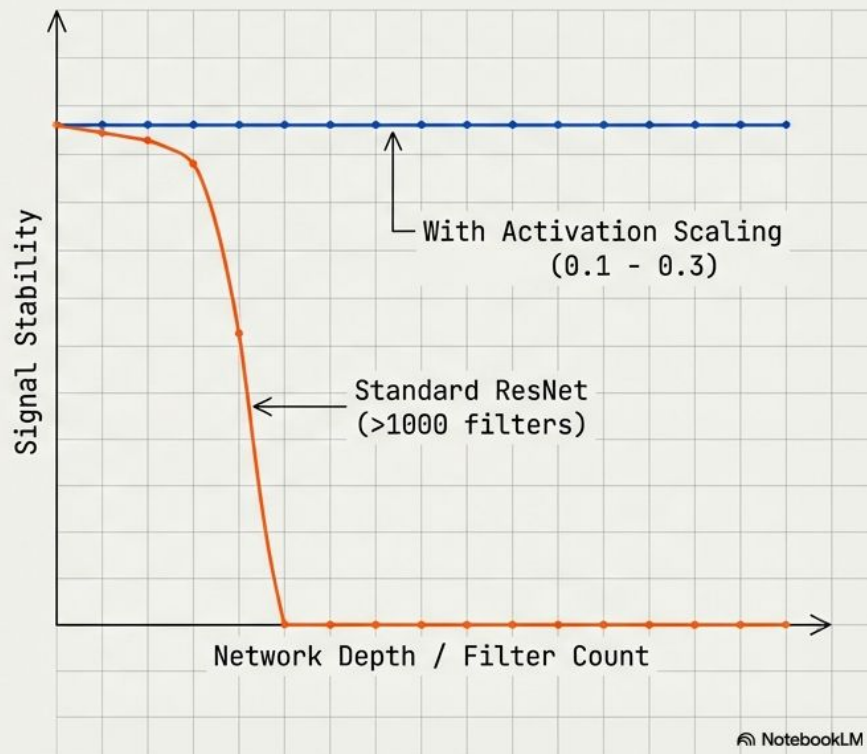
Sex estimation



Engineering Challenges: Instability in Deep Hybrids

Problem: Inception-ResNet variants with >1000 filters tend to “die” (produce zeros) early in training.

Solution: Activation Scaling. Scaling residuals by a factor of 0.1 to 0.3 before summation stabilizes the signal in high-dimensional space.



Case Study : Comprehensive review of ResNet, U-Net, DETR, and inception variants

Research Paper

Inception models' unique parallel convolution approach captures multi-scale features effectively, enhancing their ability to discern diverse tumor patterns. They strike a balance between accuracy and efficiency, making them suitable for a wide range of tasks. Nevertheless, the incorporation of multiple parallel paths increases model complexity, necessitating careful resource allocation. The trade-off between multiple paths might also lead to shallower networks compared to ResNet, potentially affecting accuracy in certain scenarios.

Where Inception Fails

While Inception was a masterpiece of feature extraction, it wasn't originally designed for the chaotic, real-time demands of object detection.



(a) Handgun detection



(b) Automatic gun detection

Inception Shortcomings

The "Sliding Window" Bottleneck: Inception-based detectors often relied on checking thousands of different regions of an image to see if an object existed there.

Localized View (Context Blindness): Inception looks at an image at different scales (3x3, 5x5, etc.) simultaneously. While great for identifying a "dog," the network often loses the "big picture" of where the dog is in relation to the rest of the scene, leading to background patches being mistaken for objects.

Complexity & Latency: Inception modules are deep and complex. While they reduce parameters compared to simple stacks, the branching paths create overhead that makes achieving high frames-per-second (FPS) difficult.

Classification



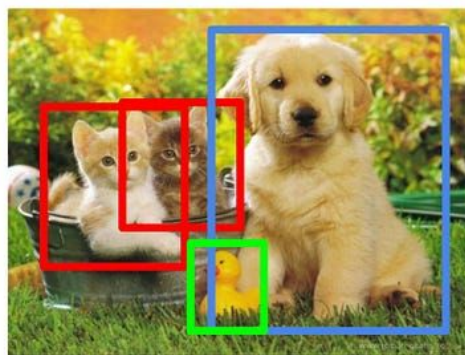
CAT

Classification + Localization



CAT

Object Detection



CAT, DOG, DUCK

Instance Segmentation



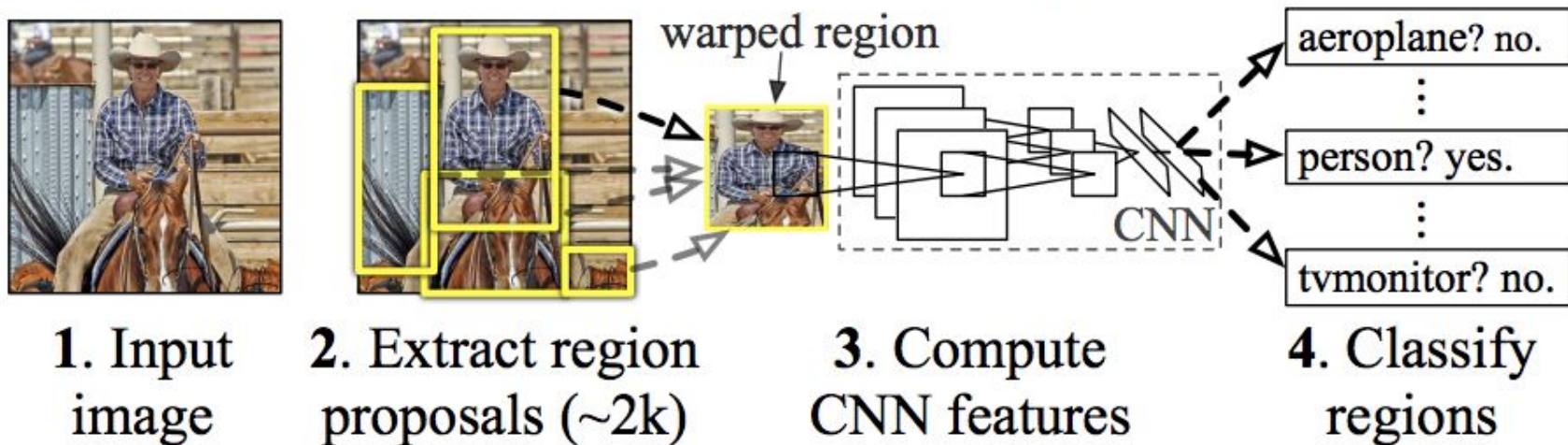
CAT, DOG, DUCK

Single object

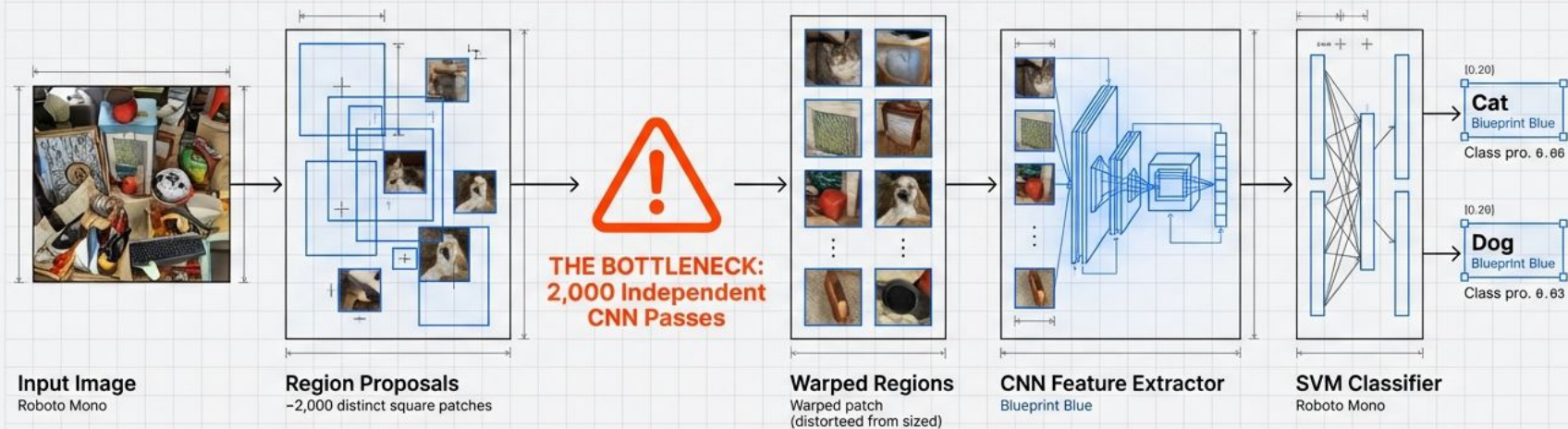
Multiple objects

R-CNN Introduction

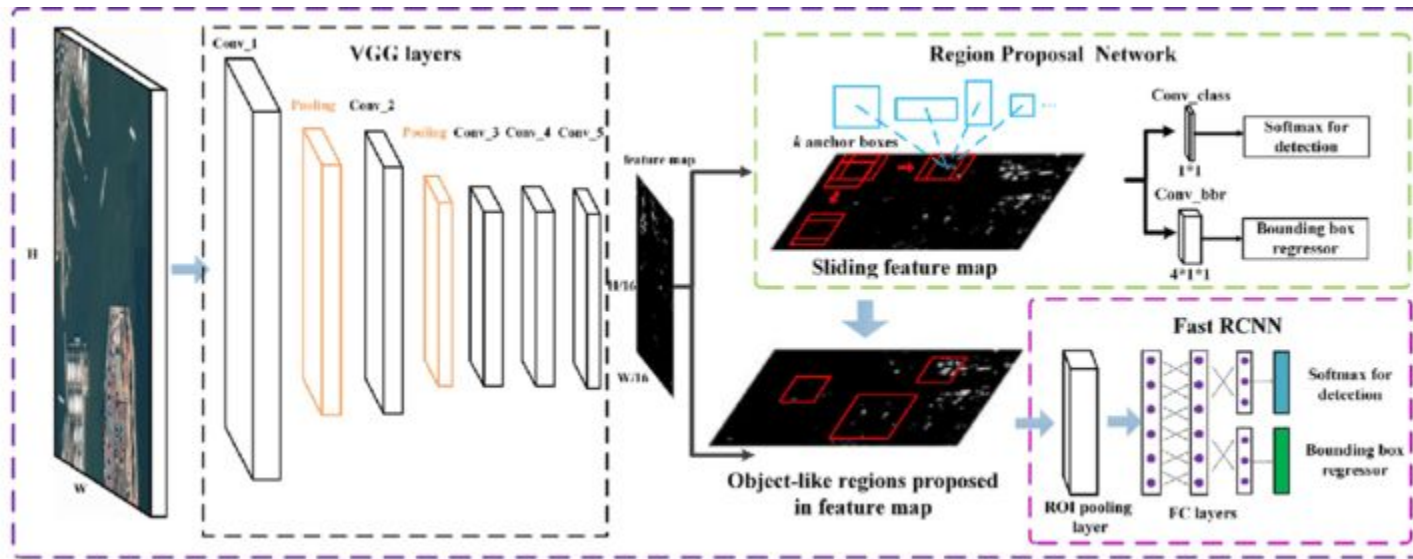
R-CNN: *Regions with CNN features*



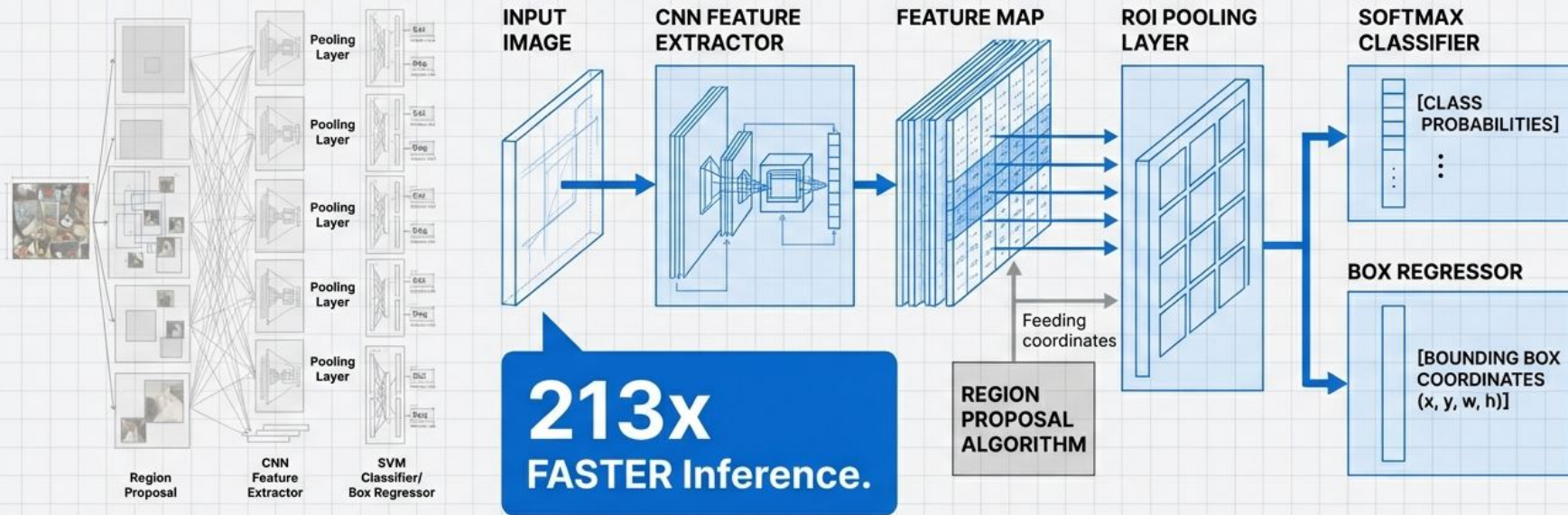
R-CNN: The Heavyweight Pioneer (2014)



Faster R-CNN Introduction



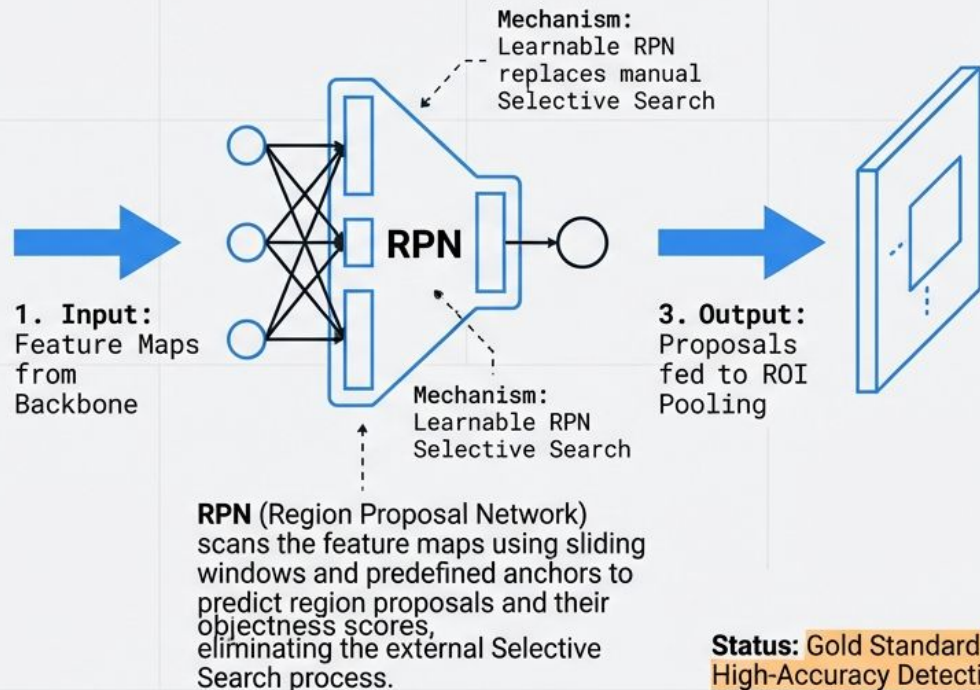
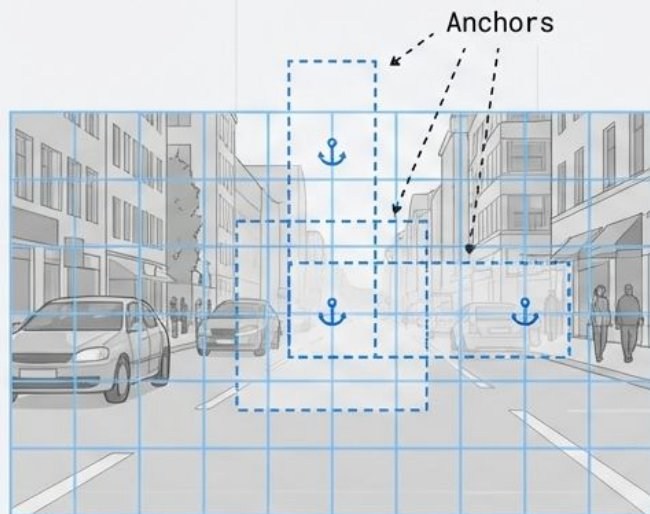
Fast R-CNN: The Speed Revolution.



PREVIOUS R-CNN (2014)
Roboto Mono

Innovation: Process the image once,
extract regions from the feature map.

Faster R-CNN: The End-to-End Standard



How R-CNN and Fast/Faster R-CNN help

The Inception Problem: Inception-based, sliding-window systems try thousands of combinations of locations and scales, causing massive computational waste.

R-CNN Solution: R-CNN introduces Region Proposals (initially Selective Search, later RPN in Faster R-CNN). Instead of sliding over every pixel, it uses a quick algorithm to find 2,000 (R-CNN) or fewer (Faster R-CNN) "Region of Interest" (RoI) candidates that are likely to contain objects.

Result: It only processes a few hundred areas rather than thousands, reducing the computation load per image drastically.

How R-CNN and Fast/Faster R-CNN help

The Inception Problem: By feeding small, cropped patches (3x3, 5x5) into a network to determine if they contain a "dog," the network loses track of where the dog is in the overall scene.

R-CNN Solution: While original R-CNN cropped regions, later iterations (Fast/Faster R-CNN) pass the entire image through the network first to create a "feature map" (using backbones like VGG or Inception). The network then extracts regions from this map.

Result: Because the feature extraction happens on the entire image before localized classification, the network retains contextual information about the "big picture," drastically reducing background patches being mistaken for objects.

How R-CNN and Fast/Faster R-CNN help

The Inception Problem: Deep, branched, Inception-style architectures are very slow at high resolutions, and managing the 2,000+ crop-and-classify steps of the original R-CNN made it unusable for real-time (FPS was very low).

R-CNN Solution: Fast R-CNN and Faster R-CNN introduced Region of Interest (RoI) Pooling. Instead of running 2,000 separate CNN passes, it runs one pass on the whole image and extracts features for all regions simultaneously. Faster R-CNN brings the RPN into the network, eliminating the slow "Selective Search" step entirely.

Result: While early R-CNN was very slow, the Faster R-CNN evolution significantly improved speed, achieving near real-time, high-accuracy, and high-FPS performance.